Bridge Gap: Complexity_Simplified_p to Complexity_Simplified_N

Puzzle 135 / secp256k1 research note. Generated 2026-06-09.
This document records what the p-side file completes, what the N-side file assumes, and the missing closure (Lambda_N, range-variable delta, carry, cubic aggregate).

1. Executive summary

Complexity_Simplified_p closes the bridge on the field side: Px_i / rx_i = Lambda (mod p). Complexity_Simplified_N copies the same numeric Lambda but the live N-side ratio is Lambda_N = Lambda + GAP, not Lambda. Until N-file uses Lambda_N and range-variable defect(d), the two files do not meet.

2. Side-by-side completion map

Layer	p-file	N-file
Root	n^3 = N (mod p)	p^3 = p (mod N)
Normalize	G/P/r x n_i^-1 -> Latin square	G/P/r x p_i^-1 (no collapse)
Collapse	CP1, CR1 constants	MISSING CQ1, Cq1
Point bridge	Px/rx = Lambda (mod p) OK	Assumes Lambda; Px/rx mod N FAILS
Scaled bridge	(n/a)	Qx = Px*d; Qx = Lambda_N*qx OK
Cubic aggregate	IP/IR = Lambda^3 (mod p) OK	IQ/Iq = Lambda^3 (mod N) FAILS
Carry	a3 = (Lambda*rx3-Px3)//p	MISSING b3 on N
Range delta	fixed p-N	only global Lambda*(p-N)

3. Core constants

```
delta = p - N = 432420386565659656852420866390673177326

Lambda (p-side bridge) = 97451685862885086182458552040892158509924235661624603229050850812487253689501

Lambda_N = Px3/rx3 mod N = 107329397079532295557318141750802481743365519021307735330654746890138631346025

GAP = Lambda_N - Lambda mod N = 9877711216647209374859589709910323233441283359683132101603896077651377656524

Px3/rx3 mod p == Lambda ? True
Px3/rx3 mod N == Lambda ? False
Qx3 == Lambda*qx3 mod N ? False
Qx3 == Lambda_N*qx3 mod N ? True
```

4. N-side BRIDGE block (to add)

```
BRIDGE_N:
  Qx_i / qx_i = CQ1/Cq1 = Lambda_N   (mod N)
  Qx_i = Lambda_N * qx_i             (mod N)

Where:
  Qx = Px * delta,   qx = rx * delta,   delta = p - N
  Lambda_N = Lambda + GAP
  GAP = Px3*rx3^-1 - Lambda            (mod N)

NOT Lambda alone. The p constant is the shadow; Lambda_N is the live N bridge.
```

5. Three corner deltas (Puzzle 135 lane)

Dual bands: d in [2^134, 2^135-1]; mirror [N-(2^135+1), N-2^134]. Mirror ceiling is +2 below naive N-(2^135-1). No delta at N+2^134 (outside the lane; that anchor is not used).

```
delta_B = p - (N - 2^135)       = 43988563352445782980164370617657004243694
delta_C = p - (N - 2^134)       = 22210491869505721318508395742023838710510
delta_D = p - (N - (2^135+1))   = 43988563352445782980164370617657004243695
```

```
G_low = (N - 2^134) - 2^134 = N - 2^135 =
115792089237316195423570985008687907809281421313194781059293213390251830427969
G_high = mirror_lo - TOP = N - 2^136 =
115792089237316195423570985008687907765725278347314657735981263638985499361601

Interior (if d known):
  defect(d) = delta + d = p - (N - d)
  new_N(d) = N - d

Relative offset (gap delta minus defect):
  at floor d=2^134: offset = 2^134
  at ceiling via G_high: offset = 2^135 + 1
  delta_D - defect(TOP) = 2 (mirror +2)
```

6. Heaven carry (p) and N gap

```
p-side carry:
  a3 = (Lambda*rx3 - Px3) // p = 21882023022690643225957990962827778287572171943979549177037792017938914896823
  Lambda*rx3 - Px3 = a3 * p exactly (mod p residue 0)

N-side carry (not yet in N-file):
  b3 = (Lambda_N*qx3 - Qx3) // N   [to be computed at puzzle height]

The integer lift across p is what lets the p-bridge close; N needs the parallel lift.
```

7. Cubic aggregate failure on N

```
IP/IR mod p == Lambda^3 mod p ? True
IQ/Iq mod N == Lambda^3 mod N ? False
IQ/Iq mod N == Lambda_N^3 mod N ? False

N-file lists Lambda^3*(p-N) but never closes IQ/Iq. Cubic on N needs a range-aware replacement, not a direct
copy of p-side Lambda^3.
```

8. Orphans in N-file (unconnected)

```
Cq = RQ*Rq^-1 mod N = 38206281270914538590302665768985461145665603420844150685897135938566415594777
  equals Lambda? False
  equals GAP? False
  equals Lambda_N? False

p3 defect root branch: C3 = Px3/p3 mod N =
31364000942182736358370866274770775676862698943039335528366879013151855459557

These aggregates exist in N-file but are not wired to Lambda_N or to d in [2^134,2^135).
```

9. Scalar reduction (tool note)

Separate from the bridge files but relevant to multiplier checks: correct rule is (k mod N)*G. Example k = 16*(N+1) = 16N+16 has k mod N = 16, so k*G = 16G. Tools that treat k>=N as infinity in widening blocks (2N,4N,8N,16N) are wrong; only multiples of N exactly map to O.

10. Closure sentence

```
p-side:  Px = Lambda * rx          (mod p)     Lambda fixed
N-side:  Qx = Lambda_N * qx        (mod N)     Lambda_N = Lambda + GAP
Height:  defect(d) = delta + d,    new_N(d) = N - d
Scalar:  only (k mod N) matters;   16(N+1) -> residue 16
```

11. Priority actions

1. Add BRIDGE_N section to Complexity_Simplified_N with Lambda_N and GAP.

2. Add four corner deltas and defect(d) = delta + d for Puzzle 135 lane.

3. Derive N-side carry b3 parallel to p-side a3.

4. Fix or replace cubic aggregate on N (IQ/Iq); do not assume Lambda^3.
5. Wire p3 branch (C3, D3) through Lambda_N, not global Lambda.
6. Regenerate search anchors from Lambda_N + relative offset, not fixed p-file Lambda.